ARTIFICIAL INTELLIGENCE

TOPIC 5.2

Attention and transformers

Explanatory study notes

Theory, worked examples and revision questions

What you should be able to do

Explain queries, keys and values; calculate a small attention output and tensor shapes; distinguish learned parameters from attention weights; explain positions, masks, encoder/decoder families and contextual embeddings; count simple projection parameters and image patches.

Based on the teacher PDFs supplied in `Lecture Slides`. The notes explain and reorganize that material. Worked calculations are study examples derived from the slide concepts, unless explicitly identified as a slide exercise. Clarifications of ambiguous or incorrect expressions are marked in context. References use **PDF page numbers**, counted from 1, rather than printed slide numbers.

English edition • 24 October 2026

Contents

1 Why attention? 3

2 Queries, keys and values 3

2.1 Shapes are a useful error check 4

2.2 Why divide by the square root? 4

3 Small attention calculations 4

3.1 The slide's weighted-sum example 4

3.2 A two-position check 5

3.3 Projection parameter count 5

4 Order: positional information 5

5 Tokenization, embeddings and context 6

6 Inside a transformer block 6

6.1 Multi-head attention 6

6.2 Residual connections and layer normalization 6

6.3 The feed-forward sublayer 7

6.4 Readout 7

7 Three transformer families 7

7.1 Next-token prediction 7

7.2 Causal masking 8

7.3 Cross-attention 8

8 Computation and memory 8

9 Vision transformers 9

10 Worked classroom exercise: A or B? 9

11 Revision questions and answers 10

11.1 Questions 10

11.2 Answers 10

12 Source guide and connection to Lab 5 10

1 Why attention?

Teacher slides: 5.2. Transformers, pp. 3–8.

An RNN transfers context through a sequence of hidden-state updates. A local convolution combines inputs within a receptive field. Attention lets an output directly combine information from selected input positions, with weights determined by the current data.

For captioning, the representation useful for one output word may differ from that needed for the next word. Rather than average all frame features once, attention computes a new weighted combination for each output query. A large weight means that a value vector contributes strongly to that particular output.

Full self-attention connects all allowed positions in one layer. Its short interaction paths help model distant relationships. Its dense pairwise computation is expensive for long inputs, and it supplies weaker built-in assumptions about spatial locality than a CNN.

Parallel training does not mean parallel generation

Transformer training can process known sequence positions together. Autoregressive generation still predicts a token, appends it, and repeats. Also, transformers have architectural assumptions and positional mechanisms: they are not models without any inductive bias.

2 Queries, keys and values

Teacher slides: 5.2. Transformers, pp. 9–14.

Use the slide's matrix convention: token embeddings are *rows* of $X \in \mathbb{R}^{N\times D}$. For one head:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K \in \mathbb{R}^{D\times d_k}$ and $W_V \in \mathbb{R}^{D\times d_v}$. Biases may be included in an implementation; omit them unless a counting question specifies them.

Quantity	Role
Query q_n	What position n is looking for.
Key k_m	What position m offers for matching.
Value v_m	Information from position m to combine into the output.
Score $q_n k_m^\top$	Compatibility of one query with one key.
Weight a_{nm}	Normalized contribution of value m to output n.

Scaled dot-product attention is

$$S = \frac{QK^\top}{\sqrt{d_k}}, \qquad A = \mathrm{softmax}_{\mathrm{row}}(S), \qquad Z = AV.$$

For one row,

$$a_{nm} = \frac{\exp(s_{nm})}{\sum_j \exp(s_{nj})}, \qquad z_n = \sum_m a_{nm} v_m.$$

Weights are nonnegative and sum to one across the keys for each query. Columns need not sum to one, and A need not be symmetric. Different query/key projections and row normalization generally prevent symmetry.

2.1 Shapes are a useful error check

Quantity	Shape
X	$N \times D$
Q, K	$N \times d_k$
V	$N \times d_v$
S, A	$N \times N$
Z	$N \times d_v$

The dot product needs queries and keys of equal feature dimension. The value dimension can differ. For cross-attention with N_q queries and N_k source tokens, A is $N_q \times N_k$ and the output has N_q rows.

Two different meanings of weights

W_Q, W_K, W_V are learned parameters shared across positions. The attention coefficients A are computed from the current input. They change when the input changes, even when model parameters are fixed. Increasing N enlarges A, but does not enlarge these projection matrices.

2.2 Why divide by the square root?

Under the illustrative assumption of independent, zero-mean, unit-variance components, a query–key dot product has variance d_k. Dividing by $\sqrt{d_k}$ keeps its scale controlled. Otherwise large scores can make softmax extremely peaked, with small gradients. The division does not itself make the row sum to one; softmax performs that normalization.

3 Small attention calculations

Teacher slides: 5.2. Transformers, pp. 11–13.

3.1 The slide's weighted-sum example

Page 11 gives query $(1, 0)$, keys $(1, 0)$, $(0, 1)$, $(2, 1)$ and values $(1, 0)$, $(0, 2)$, $(3, 1)$. Its simplified, unscaled scores are $(1, 0, 2)$ and the given softmax weights are approximately $(0.245, 0.090, 0.665)$. Thus

$$z_1 = 0.245(1, 0) + 0.090(0, 2) + 0.665(3, 1) = (2.240, 0.845).$$

Input 3 contributes most strongly because its key has the highest score with this query. Other outputs reuse the keys/values but use their own queries.

This example intentionally uses the unscaled scores shown on p. 11. Standard scaled attention on p. 13 would divide by $\sqrt{2}$ first and produce different weights. Do not mix the two conventions within one calculation.

3.2 A two-position check

If a row's scores are equal and both positions are allowed, its weights are $(0.5, 0.5)$. With values $(2, 0)$ and $(0, 4)$, output is $(1, 2)$. If the second position is masked out, its weight is zero and the output becomes $(2, 0)$.

3.3 Projection parameter count

For input width D, the three bias-free projections have

$$N_{\mathrm{QKV}} = 2Dd_k + Dd_v.$$

If $D = 4$ and $d_k = d_v = 2$, this is $3(4 \cdot 2) = 24$. Adding one bias per projected output component adds $2d_k + d_v = 6$. This count excludes an output projection, feed-forward network, embeddings and normalization.

4 Order: positional information

Teacher slides: 5.2. Transformers, pp. 15–17.

Unmasked self-attention without positional information is permutation-equivariant: permuting input rows permutes the output rows in the same way. It does not learn which token occupies which absolute position merely from the attention operation.

For example, the same occurrence of *man* among the same token multiset has no positional cue distinguishing who eats whom. Adding a position vector p_n to token embedding x_n changes this:

$$\widetilde{x}_n = x_n + p_n.$$

Mechanism	Main idea and limitation
Fixed sinusoids	Sine/cosine pairs at different frequencies encode position without learned table entries. Defined at new positions, but successful length extrapolation is not guaranteed.
Learned absolute positions	A table of $N_{\max}D$ learned values. Positions beyond the table need an extension strategy.
Relative positions	Attention incorporates positional offsets, emphasizing relationships between positions.
RoPE	Position-dependent rotations of queries/keys make their dot products depend on relative displacement as well as content.
ALiBi	A distance-dependent bias penalizes attention scores according to the head's rule.

The standard sinusoidal form is

$$p_{n,2j} = \sin\left(\frac{n}{10000^{2j/D}}\right), \qquad p_{n,2j+1} = \cos\left(\frac{n}{10000^{2j/D}}\right).$$

For the exam, understand its purpose and that it has no trainable position table; long trigonometric computations are unnecessary. A causal mask also changes the attention structure, but position encodings remain important in the architectures discussed here.

5 Tokenization, embeddings and context

Teacher slides: 5.2. Transformers, pp. 18–22.

Tokenization turns text into a sequence of vocabulary indices. A token need not be one word. Character tokenization uses a small vocabulary but long sequences; whole-word vocabularies struggle with unfamiliar words and inflections. Subword methods balance these issues by splitting uncommon words into smaller pieces.

Byte-pair encoding repeatedly merges frequent adjacent symbols during tokenizer construction. Once used for model training/inference, the vocabulary and tokenization rules define what the input indices mean.

A learned embedding table with V_{ocab} tokens and width D contains $V_{\text{ocab}}D$ parameters. Looking up a token index selects a row; a one-hot multiplication is an equivalent mathematical description, not a requirement to store large one-hot arrays.

The initial embedding of a token is context-independent. The token *bank* uses the same table entry in a river sentence and a money sentence. Its *contextual* representation after attention can differ because it combines information from different surrounding tokens.

For $V_{\text{ocab}} = 100, D = 8$, the table has 800 parameters. A sequence of length 20 produces a 20×8 embedding activation, not a 100×8 sequence activation. Vocabulary size, sequence length and feature width are different quantities.

6 Inside a transformer block

Teacher slides: 5.2. Transformers, pp. 23–29, 34.

6.1 Multi-head attention

Each head uses its own query/key/value projections. Its attention pattern can emphasize different relationships. Head outputs are concatenated and projected back to the model width:

$$\text{MHA}(X) = \text{Concat}(Z_1, \ldots, Z_h)W_O.$$

If each value head has width d_v, concatenation has width hd_v, and W_O has shape $hd_v \times D$. In the common choice $d_k = d_v = D/h$, the four combined bias-free projections contain $4D^2$ weights. Increasing the number of heads while keeping D fixed need not multiply this count; each head becomes narrower.

6.2 Residual connections and layer normalization

The slides show the original post-normalization order:

$$U = \text{LN}(X + \text{MHA}(X)), \qquad Y = \text{LN}(U + \text{FFN}(U)).$$

Residual addition requires matching shapes and provides a direct information/gradient path. Layer normalization typically normalizes each token across its feature coordinates:

$$\text{LN}(x)_j = \gamma_j \frac{x_j - \mu_x}{\sqrt{\sigma_x^2 + \epsilon}} + \beta_j.$$

Its statistics do not come from averaging across the mini-batch. It normally recomputes them from the token at both training and inference. The slide's shorthand comparison with batch normalization should not be read as saying they use the same axes or running statistics. Other transformer designs use pre-normalization; identify the order shown in a particular diagram.

6.3 The feed-forward sublayer

The same small MLP is applied independently at every token:

$$\mathrm{FFN}(x) = a(xW_1 + b_1)W_2 + b_2.$$

Attention mixes information between tokens; this MLP mixes feature coordinates within each token and adds nonlinear processing. It does not directly read another token.

With hidden width F, its count including biases is $DF + F + FD + D$. For $D = 4, F = 8$, that is $32 + 8 + 32 + 4 = 76$. Reusing the MLP at 10 or 100 positions does not change the count. Stacked transformer blocks, in contrast, ordinarily have different learned weights.

6.4 Readout

An encoder outputs one contextual vector per token. A token-level head can predict an entity label at each position. For a sequence-level task, a special CLS token can collect information through attention; a classifier then uses its final representation. CLS is a learned special input representation, not the answer label supplied in advance.

7 Three transformer families

Teacher slides: 5.2. Transformers, pp. 18, 28–35.

Family	Access to context	Typical use in the slides
Encoder-only	Bidirectional self-attention over observed input; no causal restriction.	BERT: contextual features, classification, retrieval, masked-token prediction.
Decoder-only	Causal self-attention over the prefix.	GPT-style autoregressive next-token generation.
Encoder–decoder	Encoder self-attention; decoder causal self-attention and cross-attention to the source.	Translation, captioning and speech-to-text; original Transformer, T5, Whisper.

An encoder can still use masks for padding or other restrictions. *No mask* in the family comparison means no *causal* mask, not that every implementation attends to padding.

7.1 Next-token prediction

An autoregressive model factorizes

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}).$$

Its training objective is the negative log-probability of the true next tokens. With known training sequences, targets can be shifted by one position and many predictions computed in one pass. At generation, each newly sampled or selected token extends the prefix.

7.2 Causal masking

For an input token at position n predicting the next token, allow keys at positions $m \leq n$ and block $m > n$. Add $-\infty$ to blocked scores before softmax:

$$M_{nm} = \begin{cases} 0 & m \leq n, \\ -\infty & m > n. \end{cases} \qquad A = \mathrm{softmax}_{\mathrm{row}}(S + M).$$

Blocked weights become zero. Replacing a forbidden score by zero is not the same operation: $\exp(0) = 1$, so it could still receive weight.

For three positions, the allowed pattern including the diagonal is $\left[\begin{smallmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 1 \end{smallmatrix}\right]$. There are $N(N+1)/2$ allowed pairs, so masking alone still leaves quadratic growth. Causality prevents target leakage; it does not eliminate exposure bias from using true prefixes during training and generated prefixes at inference.

7.3 Cross-attention

Queries come from the decoder representation; keys and values come from the encoder output. An output token can therefore consult the whole observed source while still being forbidden from looking at future target tokens. With 3 target positions and 5 source positions, one cross-attention matrix is 3×5.

8 Computation and memory

Teacher slides: 5.2. Transformers, p. 36.

Dense attention forms N^2 scores per head and layer. Its pairwise operations cost order $N^2 d$ for fixed feature widths; a materialized score/weight matrix uses order N^2 memory. Doubling sequence length multiplies the number of entries by four, even though the projection parameters stay fixed.

For $N = 100$, one matrix has 10,000 entries. At four bytes each, this is 40,000 bytes before accounting for other activations, heads, layers and gradients. For $N = 200$, the same calculation gives 160,000 bytes.

During causal autoregressive generation with unchanged parameters, past keys and values can be cached. The new token needs new projections and attention to that cache. Cache size grows linearly with context length for fixed architecture, but each new query still has to process its allowed past context.

FlashAttention reorganizes exact attention computation into tiles to avoid materializing the full score matrix in high-bandwidth memory; it is not inherently a sparse approximation. Sparse or sliding-window attention changes which pairs interact. Grouped-query or multi-query attention shares key/value projections between query heads and reduces cache requirements. These are different efficiency mechanisms.

9 Vision transformers

Teacher slides: 5.2. Transformers, pp. 37–40.

Split an image into patches, flatten each patch and project it to an embedding. Add positional information, optionally prepend a CLS token, then apply transformer blocks.

For image height H, width W, channels C, and nonoverlapping square patches of side P dividing both dimensions:

$$N_{\text{patch}} = \frac{H}{P}\frac{W}{P}, \qquad D_{\text{patch}} = P^2C.$$

A biased linear projection to width D has $(P^2C)D + D$ parameters, shared across patches.

For a 32×32 RGB image and 8×8 patches, there are 16 patches, each with 192 scalar inputs. With $D = 12$, the projection has $192(12) + 12 = 2316$ parameters. Adding CLS gives 17 tokens and a 17×17 attention matrix per head. A learned absolute position table would add $17(12)$ values in this setup.

Halving patch side length at fixed image size gives four times as many patches and roughly sixteen times as many pairwise attention entries, ignoring CLS. Smaller patches preserve finer detail but increase cost. Hybrid models can use convolutional tokenization or hierarchical downsampling.

10 Worked classroom exercise: A or B?

Teacher slides: 5.2. Transformers, pp. 41–42.

The slide simplifies attention by setting $W_Q = W_K = W_V = W$ and replacing softmax normalization with division by 3. Its initial row embeddings are $e_{\text{eat}} = (0, 0, 1)$, $e_{\text{red}} = (0, 0, 0)$, $e_{\text{apple}} = (1, 1, 1)$, and

$$W_A = I_3, \qquad W_B = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}.$$

Use the row-vector convention $Q = K = V = EW$ and the order (eat, red, apple). We calculate only the apple output.

For W_A, its query is $(1, 1, 1)$, scores against those three keys are $(1, 0, 3)$, and divided scores are $(1/3, 0, 1)$. Hence

$$z^A_{\text{apple}} = \tfrac{1}{3}(0, 0, 1) + 0(0, 0, 0) + 1(1, 1, 1) = (1, 1, 4/3).$$

For W_B, the projected values/keys are $(1, 0, 0)$, $(0, 0, 0)$ and $(2, 2, 1)$. The apple scores are $(2, 0, 9)$, yielding

$$z^B_{\text{apple}} = \tfrac{2}{3}(1, 0, 0) + 3(2, 2, 1) = (20/3, 6, 3).$$

Under the toy exercise's interpretation of the final axes as technology, nature and food, A emphasizes food in the eating context. This does not establish that A is universally a better learned projection: a general representation can change basis, and quality ultimately depends on an objective and evaluation.

This exercise is not standard normalized attention

Dividing scores by 3 does not force their sum to one or make them nonnegative in general. It replaces the normal softmax operation solely for the exercise. Also, using column-vector embeddings changes the order/transposes of the matrix multiplications; state a convention before calculating.

11 Revision questions and answers

11.1 Questions

1. Which determines attention weights: values, or query–key compatibility?
2. Must every column of the attention matrix sum to one?
3. For $N = 5, D = 8, d_k = 2, d_v = 3$, what are the shapes of Q, A and Z?
4. What is the purpose of dividing scores by $\sqrt{d_k}$?
5. With weights $(0.25, 0.75)$ and scalar values $(2, 6)$, what is the output?
6. Why does plain unmasked self-attention need positional information?
7. Do the MLP weights differ at every token position in one block?
8. Why must a next-token decoder mask future positions during training?
9. Where do queries, keys and values come from in encoder–decoder cross-attention?
10. Does doubling sequence length double the projection parameter count? What happens to N^2?
11. How many 4×4 patches are in a 16×16 image, and how many tokens with CLS?
12. Does a KV cache remove the need to generate successive tokens sequentially?

11.2 Answers

1. Query–key compatibility determines the weights; values supply the content being combined.
2. No. Each query row sums to one over its allowed keys.
3. $Q : 5 \times 2$, $A : 5 \times 5$, $Z : 5 \times 3$.
4. Control score scale as key dimension grows and reduce softmax saturation.
5. $0.25(2) + 0.75(6) = 5$.
6. Without positional cues it is permutation-equivariant and cannot distinguish positions by itself.
7. No; the same positionwise MLP is reused across tokens.
8. Those future tokens contain prediction targets and would leak the answers.
9. Queries from the decoder; keys and values from encoder outputs.
10. Projection count stays fixed; attention matrix entries multiply by four.
11. 16 patches and 17 tokens with CLS.
12. No; it reuses previous computations, while each new token still depends on the generated prefix.

12 Source guide and connection to Lab 5

`5.2. Transformers.pdf`: motivation (pp. 3–8), attention equations and examples (pp. 9–14), positions (pp. 15–17), tokenization/embeddings (pp. 18–22), blocks and encoders (pp. 23–29),

decoding/masks/cross-attention (pp. 30–35), efficiency (p. 36), vision transformers (pp. 37–40), and the A/B exercise (pp. 41–42).

For the Lab 5 connection, be able to explain what each matrix represents, which quantities are learned versus computed, how context changes a token representation, and why masks change which information is available. Correct dimensions and a clear interpretation are more useful than memorizing a large matrix multiplication.